CD LAB 10

Name: Abhinav Singh Bhagtana

Roll no: 29

Reg no: 230905225

Q1. Write a bison program, to check a valid declaration statement.

```
%{
#include <stdio.h>
#include <stdlib.h>

void yyerror(const char *s);
int yylex();
%}

%token INT FLOAT CHAR DOUBLE VOID IDENTIFIER NUMBER
%token COMMA SEMICOLON ASTERISK EQUALS

%%

declaration:
    type_specifier init_declarator_list SEMICOLON { printf("Valid C declaration found!\n"); }
    ;

type_specifier:
    INT | FLOAT | CHAR | DOUBLE | VOID
    ;
```

```
init_declarator_list:
   init_declarator
   | init_declarator COMMA init_declarator_list
   ;

init_declarator:
   declarator
   | declarator EQUALS NUMBER
   ;

declarator:
   pointer direct_declarator
   | direct_declarator
   ;

pointer:
   ASTERISK
   | ASTERISK pointer
   ;

direct_declarator:
   IDENTIFIER
   ;

%%
```

```
void yyerror(const char *s) {
   fprintf(stderr, "Syntax Error: %s\n", s);
}


int main() {
   printf("Enter a C declaration: ");
   return yyparse();
}
```

Lexer.l

```
%{ #include "parser.tab.h" #include <stdlib.h> #include <string.h> %}

%% "int" { return INT; } "float" { return FLOAT; } "char" { return CHAR; } "double" { return DOUBLE; } "void" { return VOID; }

[a-zA-Z_][a-zA-Z0-9_]* { return IDENTIFIER; } [0-9]+ { return NUMBER; }

"," { return COMMA; } ";" { return SEMICOLON; } "*" { return ASTERISK; } "=" { return EQUALS; }

[ \t\n] ; /* Skip whitespace */ . { printf("Unknown character: %s\n", yytext); }

%%

int yywrap() { return 1; }
```

Output:

```
10/q1$ ./declaration_parser
Enter a C declaration: double num1,num2;
Valid C declaration found!
CD_A1@CL3-26:~/Compiler Design Lab/CDLab/college/Compiler Design Lab/week
10/q1$
```

```
10/q1$ ./declaration_parser
Enter a C declaration: int a,b,sum;
Valid C declaration found!
CD_A1@CL3-26:~/Compiler Design Lab/CDLab/college/Compiler Design Lab/week
10/q1$
```

Q2.Write a bison program to check a valid decision making statements.

%{ #include <stdio.h> #include <stdlib.h>

void yyerror(const char *s); int yylex(); %}

/* Token Definitions */ %token IF ELSE SWITCH CASE DEFAULT %token LPAREN RPAREN LBRACE RBRACE COLON SEMICOLON %token IDENTIFIER NUMBER %token BREAK

/* Precedence to solve the Dangling Else ambiguity */ %nonassoc LOWER_THAN_ELSE %nonassoc ELSE

%%

/* The start symbol can be a single statement or a list of them */ program: statement_list ;

statement_list: statement | statement_list statement ;

statement: selection_statement | expression_statement | compound_statement | jump_statement | labeled_statement ;

/* Decision Making Logic */ selection_statement: IF LPAREN expression RPAREN statement %prec LOWER_THAN_ELSE { printf("Parsed: If statement\n"); } | IF LPAREN expression RPAREN statement ELSE statement { printf("Parsed: If-Else statement\n"); } | SWITCH LPAREN expression RPAREN LBRACE labeled_statement_list RBRACE { printf("Parsed: Switch statement\n"); } ;

labeled_statement_list: labeled_statement | labeled_statement_list labeled_statement ;

labeled_statement: CASE expression COLON statement { printf(" -> Case label\n"); } | DEFAULT COLON statement { printf(" -> Default label\n"); } ;

compound_statement: LBRACE statement_list RBRACE | LBRACE RBRACE ;

expression_statement: expression SEMICOLON | SEMICOLON ;

jump_statement: BREAK SEMICOLON { printf(" -> Break statement\n"); } ;

/* Simplified expression for demonstration */ expression: IDENTIFIER | NUMBER ;

%%

void yyerror(const char *s) { fprintf(stderr, "Grammar Error: %s\n", s); }

int main() { printf("Enter C decision statements (Ctrl+D to finish):\n"); return yyparse(); }

Lexer.l

%{ #include "parser.tab.h" %}

%% "if" { return IF; } "else" { return ELSE; } "switch" { return SWITCH; } "case" { return CASE; } "default" { return DEFAULT; } "break" { return BREAK; } "(" { return LPAREN; } ")" { return RPAREN; } "{" { return LBRACE; } "}" { return RBRACE; } ":" { return COLON; } ";" { return SEMICOLON; } [a-zA-Z_][a-zA-Z0-9_]* { return IDENTIFIER; } [0-9]+ { return NUMBER; } [\t\n] ; /* ignore whitespace */ . { printf("Unknown character: %s\n", yytext); } %%

int yywrap() { return 1; }

Output:

```
10/q2$ ./decision_parser
Enter C decision statements (Ctrl+D to finish):
switch(a){case 1: break;}
  -> Break statement
  -> Case label
Parsed: Switch statement
if(a) if(b) s1; else s2;
Parsed: If-Else statement
Parsed: If statement
CD_A1@CL3-26:~/Compiler Design Lab/CDLab/college/Compiler Design Lab/week
10/q2$
```

Q3. Write a bison program to evaluate an arithmetic expression involving operations +,-,* and /.

%{ #include <stdio.h> #include <stdlib.h>

void yyerror(const char *s); int yylex(); %}

/* Define the union to hold double values */ %union { double dval; }

/* Assign tokens and rules to the dval type */ %token NUM %token PLUS MINUS MULT DIV LPAREN RPAREN NEWLINE

/* Define Precedence: Bottom has higher priority */ %left PLUS MINUS %left MULT DIV %nonassoc UMINUS */* For unary minus like -5 */

%type exp

%%

input: | input line ;

line: NEWLINE | exp NEWLINE { printf("\tResult: %.4f\n", $1); } | error NEWLINE { yyerrok; } /* Recover from errors */ ;

exp: NUM { $$ = $1; } | exp PLUS exp { $$ = $1 + $3; } | exp MINUS exp { $$ = $1 - $3; } | exp MULT exp { $$ = $1 * $3; } | exp DIV exp { if ($3 == 0) { yyerror("Division by zero"); $$ = 0; } else { $$ = $1 / $3; } } | MINUS exp %prec UMINUS { $$ = -$2; } | LPAREN exp RPAREN { $$ = $2; } ;

%%

void yyerror(const char *s) { fprintf(stderr, "Error: %s\n", s); }

int main() { printf("Infix Double Calculator.\n"); yyparse(); return 0; }

Lexer.l

%{ #include "parser.tab.h" #include <stdlib.h> %}

%%

[0-9]*.?[0-9]+ { yylval.dval = atof(yytext); return NUM; } "+" { return PLUS; } "-" { return MINUS; } "*" { return MULT; } "/" { return DIV; } "(" { return LPAREN; } ")" { return RPAREN; } "\n" { return NEWLINE; } [\t] ; /* skip whitespace */ . { printf("Unknown character: %s\n", yytext); }

%%

int yywrap() { return 1; }

OUTPUT:

```
CD_A1@CL3-26:~/Compiler Design Lab/CDLab/college/Compiler Design Lab/week
10/q3$ ./calc
Infix Double Calculator.
1+2-3
        Result: 0.0000
7/9
        Result: 0.7778
(9*8)+(9-1)
        Result: 80.0000
CD_A1@CL3-26:~/Compiler Design Lab/CDLab/college/Compiler Design Lab/week
10/q3$
```

Q4. To validate a simple calculator using postfix notation. The grammar rules are as follows – input □

input line | ε line □ '\n' | exp '\n' exp □ num | exp exp '+'

| exp exp '-'

| exp exp '*'

| exp exp '/'

| exp exp '^'

| exp 'n'

%{

#include <stdio.h>

#include <stdlib.h>

#include <math.h>

void yyerror(const char *s);

int yylex();

```
%}

%union {
   double dval;
}

%token <dval> NUM
%token PLUS MINUS MULT DIV EXPON NEG NEWLINE
%type <dval> exp

%%

input:
   | input line
   ;

line:
   NEWLINE
   | exp NEWLINE { printf("\tResult: %.2f\n", $1); }
   ;

exp:
```

```
   NUM          { $$ = $1; }
   | exp exp PLUS  { $$ = $1 + $2; }
   | exp exp MINUS { $$ = $1 - $2; }
   | exp exp MULT  { $$ = $1 * $2; }
   | exp exp DIV   {
                  if ($2 == 0) {
                     yyerror("Division by zero");
                     $$ = 0;
                  } else {
                     $$ = $1 / $2;
                  }
               }
   | exp exp EXPON { $$ = pow($1, $2); }
   | exp NEG      { $$ = -$1; }
   ;

%%

void yyerror(const char *s) {
   fprintf(stderr, "Error: %s\n", s);
}
```

```
int main() {
   printf("Postfix Calculator (Use 'n' for unary negation)\n");
   printf("Example: 3 4 + 2 * n (Result: -14.00)\n");
   yyparse();
   return 0;
}
```

Lexer.l

```
%{ #include "parser.tab.h" #include <math.h> %}

%%

[0-9]+(.[0-9]+)? { yylval.dval = atof(yytext); return NUM; } "+" { return PLUS; } "-" { return MINUS; } "*" { return MULT; } "/" { return DIV; } "^" { return EXPON; } "n" { return NEG; } "\n" { return NEWLINE; } [ \t] ; / skip whitespace */ . { printf("Unknown character: %s\n", yytext); }

%%

int yywrap() { return 1; }
```

Output:

```
10/q4$ ./postfix_calc
Postfix Calculator (Use 'n' for unary negation)
Example: 3 4 + 2 * n (Result: -14.00)
3 6 + 4 /
        Result: 2.25
3 6 + 45 8 / *
        Result: 50.62
CD_A1@CL3-26:~/Compiler Design Lab/CDLab/college/Compiler Design Lab/week
10/q4$
```